COSC3500 - MPI

Ian McCulloch

University of Queensland
School of Mathematics and Physics

Semester 2, 2022

Message Passing Interface https://www.mpi-forum.org/

MPI *specification* – a programming interface with multiple implementations

SPMD model – Single Program, Multiple Data. Multiple copies of the program running in separate address spaces, communicating via message passing.

History

- Prehistory (1980's): Rise of distributed memory parallel computing, many incompatible software tools for writing software
- Early 1990's: Basic features of MPI standard finalized - ORNL supercomputing faculity + $\sim$ 40 organisations
- 1994: Final version of MPI-1.0
- 1998: MPI-2 – adds process management and Spawn
- 2014: MPI-3 – non-blocking collective operations, one-sided communication
- 2021: MPI-4 – 64-bit array sizes, better support for hybrid (OpenMP/CUDA/etc), error handling improvements, new way to initialize MPI
- In progress: MPI-5

Programming model

Originally MPI was designed for distributed memory architectures

(Many early implementations only allowed single threaded programming)

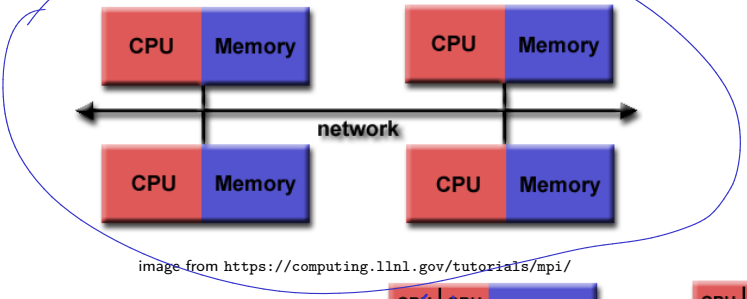
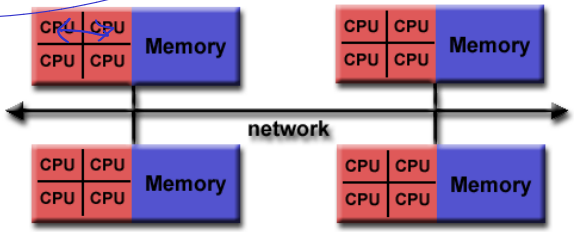


image from https://computing.llnl.gov/tutorials/mpi/



Modern paradigm: hybrid distributed memory

The *program model* remains distributed memory – no shared variables

Basic message passing

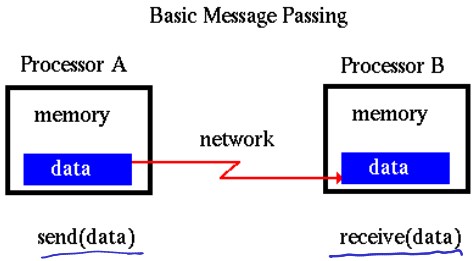


image from https://www.cs.mtsu.edu/

Proccess – an instance of an MPI program running on some node

- One process per CPU (MPI will typically use shared memory to communicate within a node)
- One process per node (Hybrid MPI/OpenMP)
- Other possibilities...

MPI *communicator* – a collection of processes that may communicate with each other. Within a communicator, processes are given a *rank* from $0 \ldots N-1$

MPI_COMM_WORLD – the predefined communicator which includes all processes

Hello world

```
#include <mpi.h>
#include <cstdio>
using namespace std;
int main(int argc, char** argv)
{
   MPI_Init(&argc, &argv);

   int world_size;
   MPI_Comm_size(MPI_COMM_WORLD, &world_size);

   int my_rank;
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

   printf("Hello from process %d out of %d\n", my_rank, world_size);

   MPI_Finalize();
}
```

N processes

rank 0, 1, ... N-1

Running the example

Goliath

```
ssh user@goliath.compute.eait.uq.edu.au
module load gcc mpi/openmpi-x86_64
```

Getafix

```
ssh user@getafix.smp.uq.edu.au -p 2022
module load gnu/10.2.1 gnutools mpi/openmpi3_eth
```

```
#git clone git://git.science.uq.edu.au/cosc3500
cd cosc3500
git pull
cd mpi
make hello
mpiexec -n 4 ./hello
```

Send and Receive

Suppose process A wants to send a message to process B

Process A

- Pack the data into a *buffer*
- Assign a message ID (known as a tag)

```
MPI_Send(
    void* data,
    int count,
    MPI_Datatype datatype,
    int destination,
    int tag,
    MPI_Comm communicator)
```

Process B

- Allocate a *receive buffer*

```
MPI_Recv(
    void* data,
    int count,
    MPI_Datatype datatype,
    int source,
    int tag,
    MPI_Comm communicator,
    MPI_Status* status)
```

MPI datatypes

Send and Recv use MPI Datatypes to specify the structure of a message.

MPI datatype	**C equivalent**
MPI_SHORT	short int
MPI_INT	int
MPI_LONG	long
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_CHAR	char
MPI_BYTE	char

- also types for unsigned, and fixed sizes such as MPI_UINT32_T
- The *count* is the number of elements of the specified datatype
- it is possible to create your own datatypes
  (sometimes useful, especially on fancy hardware)

Send/Recv example

```
// Find out rank, size
int world_size;
MPI_Comm_size(MPI_COMM_WORLD, &world_size);
int my_rank;
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

if (my_rank == 0)
{
    int number = 66;
    MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    printf("Process 0 sent number %d to process 1\n", number);
}
else if (my_rank == 1)
{
    int number;
    MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 1 received number %d from process 0\n", number);
}
```

buffer is an array of 1 int

destination process

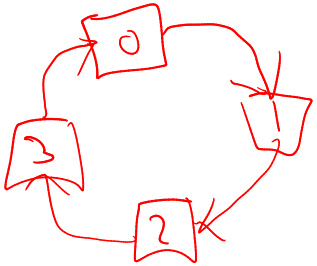
tag

from this process

```
make send-recv
mpiexec -n 2 ./send-recc
```

A more complicated example – message passing ring

```
// maximum number of messages to send
int count_limit = 4;
// rank 0 starts off
if (my_rank == 0) {
   int data = 66;
   MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
   printf("Process 0 sent number %d to process 1\n", data);
}
for (int count = 0; count < count_limit; ++count) {
   int data;
   int from = (my_rank-1+world_size) % world_size;
   MPI_Recv(&data, 1, MPI_INT, from, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
   printf("Process %d received number %d from process %d\n",
            my_rank, data, from);
   if (count < count_limit-1 || my_rank != 0) {
      int send_to = (my_rank+1) % world_size;
      data = (data*37) % 100;
      MPI_Send(&data, 1, MPI_INT, send_to, 0, MPI_COMM_WORLD);
      printf("Process %d sent number %d to process %d\n", my_rank, data,
            send_to);
   }
}
```



```
make ring
mpiexec -n 4 ./ring
```

This isn't very efficient, since only one message is in flight at a time. At any moment, $N - 1$ processes are blocked waiting for a message. We will see later how we can ovelap communication so that all processes can send and receive simultaneously using non-blocking communication.

Dynamic size buffers

using MPI_Status

```
// Allocate a sufficiently large buffer for MaxNumbers
MPI_Status status;
// Receive at most MaxNumbers from process zero
MPI_Recv(numbers, MaxNumbers, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);

// After receiving the message, check the status to determine
// how many numbers were actually received
MPI_Get_count(&status, MPI_INT, &number_amount);
```

using MPI_Probe

```
MPI_Probe(
    int source,
    int tag,
    MPI_Comm comm,
    MPI_Status* status)
```

Simlar signature to MPI_Recv – does everything but actually receive the message

Buffered vs non-buffered communication

`MPI_Send` *may* block. Once it returns you are allowed to reuse the send buffer, which means that the implementation can either block until it can send the message, *or* it could buffer the message internally and send it later.

`MPI_Ssend` (Synchronous Send) avoids the possibility of buffering by blocking until a matching receive has been posted, so it also combines some synchronisation, which may be useful in your program. The function signature is identical to `MPI_Send`.

For completeness, we mention but you don't want to use: `MPI_Bsend` (Buffered Send) copies the data into a buffer supplied using `MPI_Attach_buffer`. If the buffer is currently in use it will block until enough buffer space is available.



Non-blocking communication

Most operations in MPI have a *non-blocking* (*immediate*) variant.

```
MPI_Isend(
    void* data,
    int count,
    MPI_Datatype datatype,
    int destination,
    int tag,
    MPI_Comm communicator,
    MPI_Request* request)
```

cannot modify this!

Returns immediately without waiting for the message to be copied out from the buffer. A communication request handle is returned for handling the pending message status. The program should not modify the application buffer until subsequent calls to MPI_Wait or MPI_Test indicate that the non-blocking send has completed.

A high quality implementation will typically perform the communication asynchronously, but this is not mandated by the standard; it is possible that no communication occurs until MPI_Wait or MPI_Test is called (and older implementations worked exactly this way!)

Non-blocking receive

There is a corresponding non-blocking (immedate) receive operation.

```
MPI_Irecv(
   void* data,
   int count,
   MPI_Datatype datatype,
   int source,
   int tag,
   MPI_Comm communicator,
   MPI_Request* request)
```

Returns immediately without waiting for the message to arrive. A communication request handle is returned for handling the pending message status. The program should not access the data buffer until calls to MPI_Wait or MPI_Test indicate that the non-blocking receive has completed.

A high quality implementation will typically perform the communication asynchronously, but this is not mandated by the standard; it is possible that no communication occurs until MPI_Wait or MPI_Test is called (and older implementations worked exactly this way!)

Note: you don't need to pair up non-blocking operations. Any kind of receive can get a message from any kind of send. In MPI programs it is unusual for both send and receive to be non-blocking.

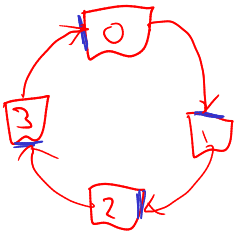
Overlapping communication and computation

We can reformulate our *ring* example using a non-blocking receive: `ring-nonblocking-recv`

```
for (int count = 0; count < count_limit; ++count) {
   // receive some data in non-blocking mode, we allow any source process and any tag
   int recv_data;
   MPI_Request request;
   MPI_Irecv(&recv_data, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &request);
   printf("Process %d initiates non-blocking receive\n", my_rank);

   // we could do some computation here, such as computing some data and sending it.
   int send_data = my_rank * count;
   int send_to = (my_rank+1) % world_size;
   int send_tag = 0;
   MPI_Send(&send_data, 1, MPI_INT, send_to, send_tag, MPI_COMM_WORLD);
   printf("Process %d sent number %d to process %d using tag %d\n",
      my_rank, send_data, send_to, send_tag);

   // Now wait for the MPI_Irecv to finish.
   // We cannot access recv_data until the recv has completed.
   MPI_Status status;
   MPI_Wait(&request, &status);
   printf("Process %d received data %d from process %d with tag %d\n",
      my_rank, recv_data, status.MPI_SOURCE, status.MPI_TAG);
}
```



Exercise: modify this code to use a standard receive and a non-blocking *send*.

Collective communication and synchronization

So far all of the communication has been point-to-point

Collective communication involves **all** processes in a communicator

- Often more convenient than point-to-point
- Might be faster (on some hardware)

Collective communication implies a synchronization point – all processes must reach the same point in the program.

```
MPI_Barrier(MPI_Comm communicator);
```

Functions similarly to a barrier in OpenMP. *All* processes must reach the barrier for execution to proceed.

Broadcast

In a *broadcast*, one process sends the same data to all processes in a communicator.

- Might be done for algorithmic reasons – eg matrix-vector multiply
- Send configuration parameters to all processes
- Avoid multiple processes reading from the same file

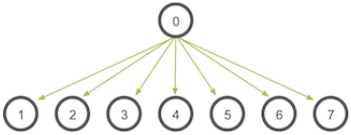


image from mpitutorial.com

```
MPI_Bcast(
    void* data,
    int count,
    MPI_Datatype datatype,
    int root,
    MPI_Comm communicator)
```

All processes have `data` and `count` parameters – this is a send buffer for the *root* process and a receive buffer for all other processes.

Broadcast example

```
// Broadcast a string to all nodes
std::string s;

// The root node can be any process in the communicator
int root_rank = world_size / 2;

// The root node initializes the string.
if (my_rank == root_rank) {
    s = "Hello COSC3500/7502!  This is the root process " + std::to_string(root_rank)
        + " pid " + std::to_string(getpid()) + " broadcasting a string";
}

// Firstly broadcast the length of the string
int Size = s.size();
MPI_Bcast(&Size, 1, MPI_INT, root_rank, MPI_COMM_WORLD);

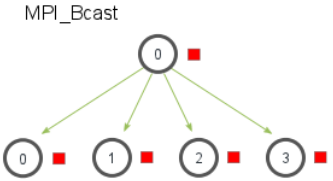
// Now ensure the destination string is the right length and broadcast the string data
s.resize(Size);
// C++17: s.data() is modifiable.
MPI_Bcast(s.data(), Size, MPI_CHAR, root_rank, MPI_COMM_WORLD);

std::cout << "Process " << my_rank << " has the string: " << s << std::endl;
```

This is designed so that the code that reconstructs the string also works on the root process, which avoids a lot of ugly `if (my_rank != root_rank) ...` constructs.

Scatter/Gather

A *Broadcast* sends the same data to all other processes.



A *Scatter* operation distributes the data to the other processes.

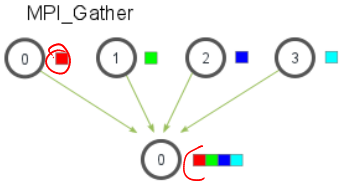
MPI_Scatter

```
MPI_Scatter(
    void* send_data,
    int send_count,
    MPI_Datatype send_datatype,
    void* recv_data,
    int recv_count,
    MPI_Datatype recv_datatype,
    int root,
    MPI_Comm communicator)
```

- `send_count` is how many elements to send to each process (often equal to number of array elements divided by the number of processes)
- Use `MPI_Scatterv` if you have different counts for each process
- In simple cases `send_datatype` and `recv_datatype` will be the same

MPI_Gather

`MPI_Gather` does the opposite of `MPI_Scatter` – collect array elements from all processes and gather them at a single process.

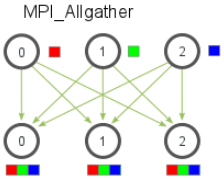



```
MPI_Gather(
    void* send_data,
    int send_count,
    MPI_Datatype send_datatype,
    void* recv_data,
    int recv_count,
    MPI_Datatype recv_datatype,
    int root,
    MPI_Comm communicator)
```

- Only the root process needs to have a valid receive buffer – other processes can set this to null
- `recv_count` is the count of elements received per process, not the total! Normally `recv_count == send_count`

Other collective operations

- `MPI_Allgather` – `MPI_Gather` followed by `MPI_Bcast`



- `MPI_Reduce` – similar to reduce in OpenMP – apply an operation (such as summation) to all elements of a distributed array
- `MPI_Allreduce` – Similar to `MPI_Reduce` followed by `MPI_Bcast`

Running batch jobs via Slurm

Never run large compute jobs on the front-end node!

Run any non-trivial computation using the SLURM batch queues.

To do this, you need to write a *job script*. You can specify the number of CPU's and the number of tasks (MPI processes) either in the job script itself or on the command line.

```
sbatch -c <number_of_cpus> my-openmp-script

sbatch -n <number_of_MPI_processes> my-mpi-script
```

Executing parallel programs with Slurm

`sbatch` batch allocates resources (memory, nodes, CPUs, GPUs, etc) and executes the *job script* on the first allocated node (primary node).

`srun` command runs a parallel job. Use within a job script for fine control over resources (eg to run a program using a subset of the resources). Use on the command line as a shortcut for an interactive `sbatch` job. For MPI programs, either `srun` or `mpiexec/mpirun` can be used to launch MPI programs.

Slurm resource management options

Node	`-N`	`--nodes=N`	A collection of CPUs sharing memory
Task	`-n`	`--ntasks=n`	A process. Maps onto MPI ranks
		`--tasks-per-node=n`	Alternative to `--ntasks`
CPU	`-c`	`--cpus-per-task=n`	
		`--cpus-per-node=n`	
Memory		`--mem=x`	Memory (per node)
		`--mem-per-cpu=x`	Memory (per CPU)

Executing MPI programs via Slurm

Job script

```
#!/bin/bash
#SBATCH --job-name=my-mpi-program
#SBATCH --cpus-per-task=1
#SBATCH --ntasks=4
#SBATCH --mem-per-cpu=1G
#SBATCH --partition=cosc          #getafix
#SBATCH --partition=coursework #goliath

module load mpi/openmpi3_eth       #getafix (or mvapich2_eth or intelmpi)
module load mpi/openmpi-x86_64     #goliath

mpiexec program.out
```

Submit the job script to the batch queue

```
sbatch script.batch
```

On getafix you will get some harmless warnings, arising because it only has ethernet and doesn't have specialized communication hardware.

Other resources

- MPI standard `https://www.mpi-forum.org`
- `https://hpc-tutorials.llnl.gov/mpi/`
- `https://mpitutorial.com/tutorials/`
- `https://www.open-mpi.org/`